

CS633 Assignment Group Number 22

Kishan Kumar Mishra (251110044)

Aditya Pushkar (251110004)

Govind Gupta (251110033)

Harsh Goel (251110034)

April 12, 2026

1 Code Logic and Implementation

1.1 3D Domain Decomposition and Layout

The global simulation space is partitioned into a Cartesian grid of processes defined by (px, py, pz) . Each process manages a local sub-domain (nx, ny, nz) .

- **Halo Regions and Total Dimensions:** To facilitate boundary computations, each sub-domain is wrapped in a "ghost" or halo region of thickness $h = d/6$. This results in a total memory allocation for each process defined by:

$$nx_{tot} = nx + 2h, \quad ny_{tot} = ny + 2h, \quad nz_{tot} = nz + 2h$$

The data structure accounts for one halo region at each boundary (left/right, bottom/top, front/back), ensuring that the actual data starts at index h and ends at $nx + h - 1$ in each respective dimension.

- **Memory Serialization:** Data is stored in a 1D contiguous double-precision array of size $(nx_{tot} \cdot ny_{tot} \cdot nz_{tot})$. The mapping from 3D space (x, y, z) to 1D index is handled by the `IDX` macro:

$$\text{IDX}(x, y, z) = x \cdot (ny_{tot} \cdot nz_{tot}) + y \cdot (nz_{tot}) + z$$

By placing the Z-dimension as the fastest-changing index, we ensure that the most frequent memory accesses are contiguous, drastically reducing cache misses during the inner-most loop execution.

1.2 Halo exchange

Each process uses `MPI_Irecv` and `MPI_Isend` operations to exchange data across all six faces with neighboring ranks. For processes located on the global grid boundaries, the neighbor rank is set to `MPI_PROC_NULL`; this ensures that the MPI calls act as immediate no-ops, allowing the communication logic to remain uniform regardless of a process's position in the grid.

1.3 The d -point Stencil Algorithm

The stencil computation approximates physical properties by averaging values of neighboring points.

- **Stencil Masking:** Not all data points have the same number of neighbors (e.g. global grid boundaries). We implement a `weight_mask` where every point in the grid is assigned a value of 1.0 (valid) or 0.0 (invalid).
- **Normalization:** To ensure the accuracy of the average, a `divisors` array is pre-computed. This array stores the number of valid neighbors for every point.

1.4 Isovalue Intersection (Voxel Analysis)

- **Voxel Definition:** A voxel is treated as a $1 \times 1 \times 1$ unit cube. We sample the values at the 8 vertices of this cube.
- **Range Testing:** A voxel is flagged as intersected if and only if:

$$\min(\text{vertices}) \leq \text{isovalue} \leq \max(\text{vertices})$$

2 Code Optimizations

2.1 Communication-Computation Overlap (Latency Hiding)

2.1.1 Stencil Computation

For the d -point stencil, the execution is orchestrated to maximize CPU utilization during data transit:

1. **Initiate Halo Exchange**
2. **Inner Core Computation:** While the network hardware handles the data transfer, the CPU computes the stencil for the "Inner Core" (points that do not depend on neighbor data).
3. **Synchronization:** Call `MPI_Waitall` to ensure the halo regions are updated.
4. **Boundary Computation:** Complete the stencil for the outer slabs (the neighbor-dependent regions).

2.1.2 Isovalue Intersection Computation

After the stencil results are swapped into the main data buffer, the isovalue analysis proceeds:

1. **Initiate Halo Exchange:** This communicates the updated stencil results to neighbour processes.
2. **Concurrent Internal Isovalues:** While the neighbor data is in transit, the process iterates through all internal voxels that are contained entirely within the local memory.

3. **Synchronization:** `MPI_Waitall` ensures that the values required for "straddling" voxels (those crossing process boundaries) are available.
4. **Boundary Isovalues:** The `compute_ngbr_isovalues` function is invoked to process the remaining straddling voxels.

This dual-layered overlap ensures that the process is rarely idle, effectively masking the communication overhead behind the high computational density of both the stencil and isovalue tasks.

2.2 Ownership and Deduplication Strategy

In a distributed environment, voxels straddling the boundary between two processes pose a risk of double-counting.

- **Right-Ordering Ownership:** We implement a strict ownership rule where a voxel is owned solely by the process with the lower rank coordinate in each dimension. For e.g. in the X dimension, a voxel shared between two processes is owned by the left process(lower coordinate).
- **Intra-Process Exclusion:** Within the `compute_ngbr_isovalues` function, we use bounds to ensure that edges and corners shared between the boundary X, Y, and Z faces are only visited once, preventing internal double counting.

2.3 Advanced MPI Datatypes (Zero-Copy Transfer)

The code avoids the overhead of manual buffer packing by using `MPI_Type_indexed`. This allows the MPI library to understand the non-contiguous layout of the 3D faces in 1D memory.

3 Results and Analysis

3.1 Execution time analysis

P	N	px	py	pz	Avg Time (s)
32	120	4	4	2	1.635027
48	120	6	4	2	1.840645
64	120	4	4	4	1.865849
96	120	6	4	4	1.917826
32	240	4	4	2	13.653677
48	240	6	4	2	15.257661
64	240	4	4	4	15.618284
96	240	6	4	4	16.022451

Table 1: Average execution time for each configuration

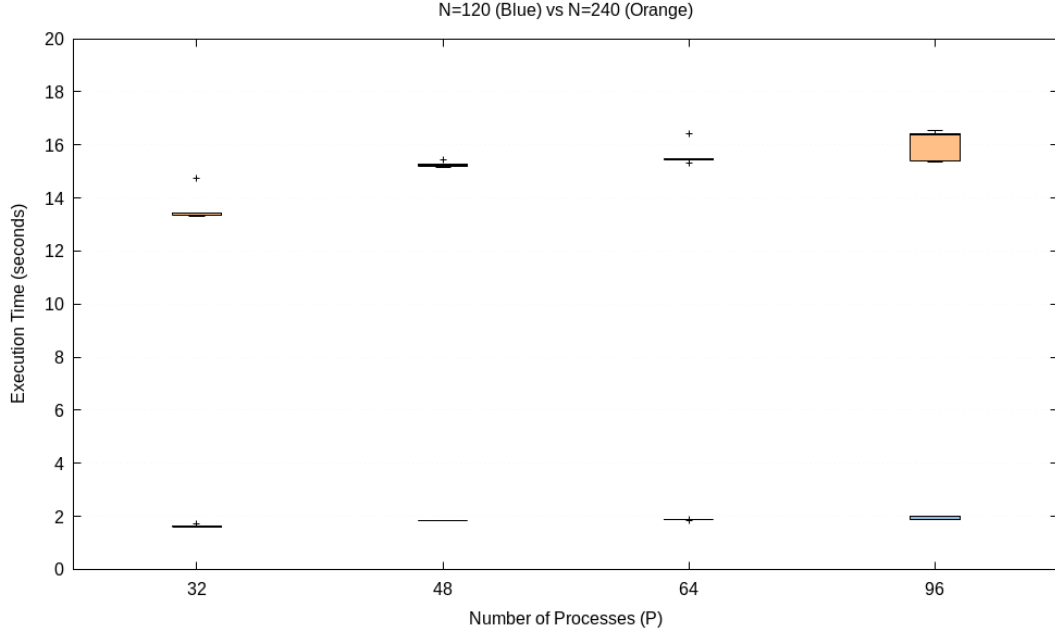


Figure 1: Execution time vs number of processes.

From Table 1, we observe that:

- **Scaling with Problem Size (N)**

When N doubles from 120 to 240, the execution time increases by a factor of approximately 8 times across all configurations. Since $2^3 = 8$, this scaling behavior strongly suggests that the algorithm follows a computational complexity of $O(N^3)$. This is a typical characteristic of 3D grid computations.

- **Scaling with Processes (P)**

As P increases from $32 \rightarrow 48 \rightarrow 64 \rightarrow 96$, the average execution time consistently increases. As more processors are added, the time saved on local calculations is eclipsed by the rising cost of inter process communication and synchronisation.

3.2 Intersected voxels count analysis

Iteration	Property 1	Property 2
1	311,071	312,561
2	79,123	80,927
3	25,938	26,272
4	13,459	13,395
5	10,807	10,864

Table 2: Isovalue intersection count for: Run=1 P=32 N=120 px=4 py=4 pz=2

For each configuration, across iterations we notice a reduction in the number of voxels intersecting with the isovalue. In each iteration, the stencil computation pushes the values towards a local mean. This reduces the range of values in a voxel, reducing its chance to intersect with the isovalue.